

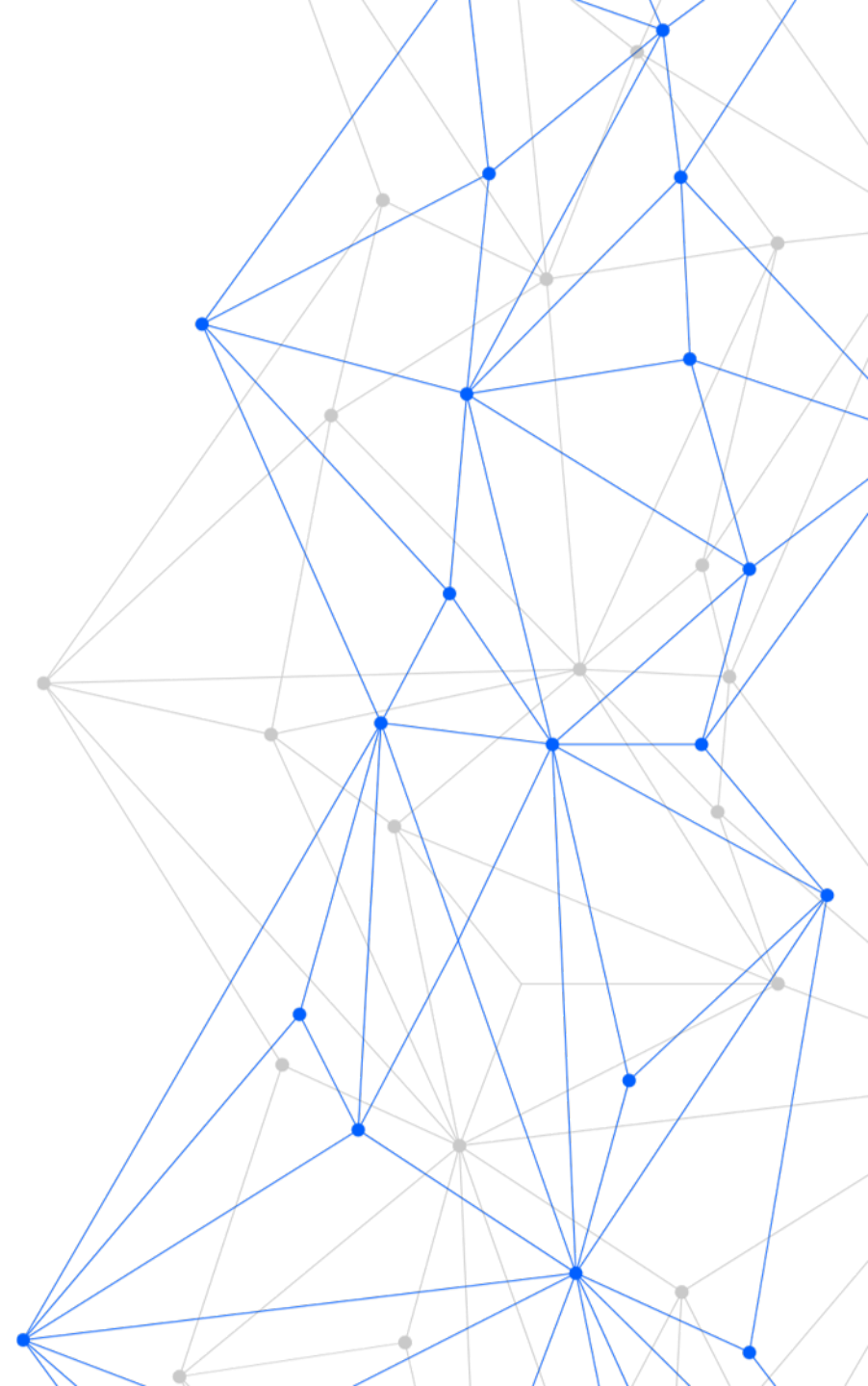
RESEARCH TALK · OCTOBER 2026

Recall and tokens in a personal AI knowledge base

What decides whether the right note reaches the model, and what it costs to get it there

Andrey Maltsev

github.com/guanchzhou/hilbert-paper



The question

An AI assistant answers from my own notes. Two numbers matter:

Recall: is the note that answers the question among what the model is given?

Cost: how many tokens does the model read to get it?

A switch that saves tokens but drops the right note is a regression, not a saving.

Rule of the study: test every available combination, measure real data, never fit results to an expected answer.

Measured on one frozen corpus

1,225 notes, 5,155 chunks
817 development questions with known answers
817 sealed held-out questions, opened once

Everything is reproducible

Every number on these slides is read from a result file in the repository.

PART 1

What was tested



Storage and search

gbrain

The personal knowledge base: notes mirrored from Obsidian into Postgres, chunked, embedded, searchable over MCP by Cursor and Claude Code.¹

Postgres + pgvector

The database. pgvector adds vector columns and an HNSW nearest-neighbour index.²

Qwen3-Embedding

Turns each chunk and each question into a 1,024-number vector; runs locally through MLX on the laptop GPU.³

Keyword search

Postgres full-text search: matches words, ranks by term density. The classic baseline next to vectors.⁴

Reranker

A small cross-encoder (Qwen3-Reranker 0.6B) that reads the question and each candidate together and re-orders them.⁵

ArangoDB

A document and graph database, tested on the same vectors as a possible second engine.⁶

¹gbrain, github.com/garrytan/gbrain, version 0.60.64.

²pgvector 0.8.6, github.com/pgvector/pgvector; HNSW: Malkov and Yashunin, IEEE TPAMI 2020.

³Zhang et al., Qwen3 Embedding, arXiv:2506.05176, 2025.

⁴PostgreSQL 16 documentation, chapter 12, Full Text Search.

⁵Nogueira and Cho, Passage re-ranking with BERT, arXiv:1901.04085, 2019.

⁶ArangoDB 3.12 documentation, vector indexes.

The key and the **agent** side

zig-hilbert and hk1

Built for this study: turns a vector into a short sortable text key on a Hilbert curve, so a database prefix range could act as a cheap search filter.¹

Evidence unit

What is handed to the model under a 6,000-token budget: a chunk, a window, a section, or the whole note.

RTK

A filter that shortens shell command output before an AI agent reads it; hooks into Cursor and Claude Code.²

Cursor CLI

The Cursor agent run headless from the terminal, with a choice of models.

Claude Code

Anthropic's coding agent, run headless with the same tasks.

Local model

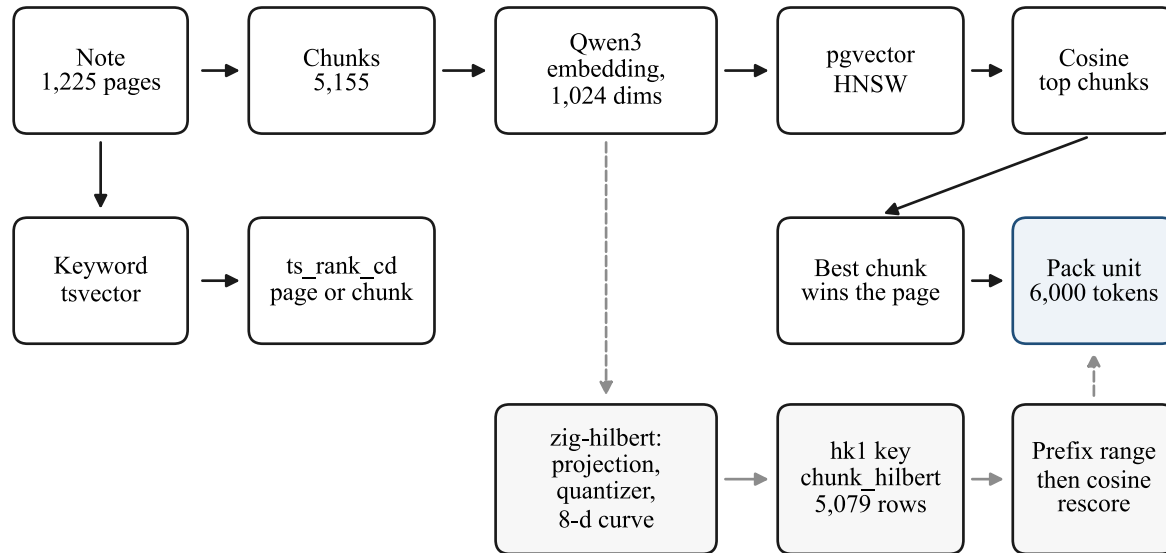
qwen3.8, a 27B model in Ollama on the laptop, with context windows from 8K to 128K tokens.³

¹Maltsev, zig-hilbert, github.com/guanchzhou/zig-hilbert, v0.2.1; Hilbert, Mathematische Annalen 38, 1891.

²RTK 0.51.0, www.rtk-ai.app.

³Ollama, ollama.com.

How it was measured



dashed: the filter under test, not used in production

Preregistered. Hypotheses and pass criteria were written down and committed before any measurement.

Paired. One switch changes at a time; every comparison is made question by question.

Corrected. Paired statistical tests with correction for testing many things at once.¹

Sealed. The held-out questions stay closed until a configuration has won on the development set.

¹Wilcoxon, Biometrics Bulletin 1945; McNemar, Psychometrika 1947; Holm, Scandinavian Journal of Statistics 1979.

PART 2

Results



Vectors beat keywords by a wide margin

Result 1

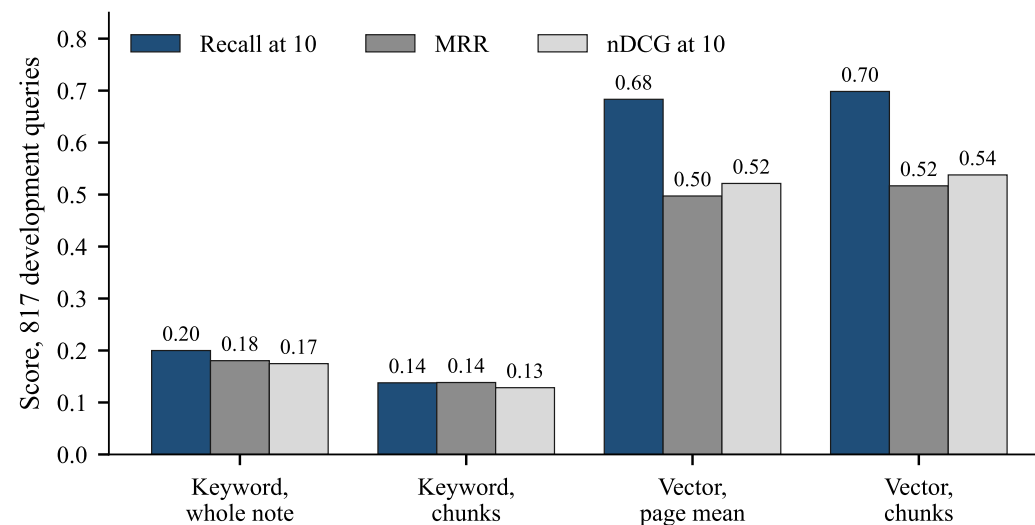
0.698

recall at 10, vector search over chunks

0.200

keyword search over whole notes

Keyword search returned nothing at all for 401 of 817 questions:
it joins every word of a long question with AND.



How the scores are computed

Each search returns a ranked list of notes; a chunk hit counts for its note. Each of the 817 questions has 1 to 7 notes known to answer it (exactly one for 584). Every score runs from 0 to 1, is computed per question, and is averaged over all 817.

Recall at 10: the share of a question's answering notes that appear in the first ten results.

MRR, mean reciprocal rank: 1 divided by the rank of the first answering note: 1 at rank 1, 0.5 at rank 2, 0 if it is not returned.¹

nDCG at 10, normalised discounted cumulative gain: each answering note in the top ten earns $1 / \log_2(\text{rank} + 1)$; the sum is divided by the best possible sum.²

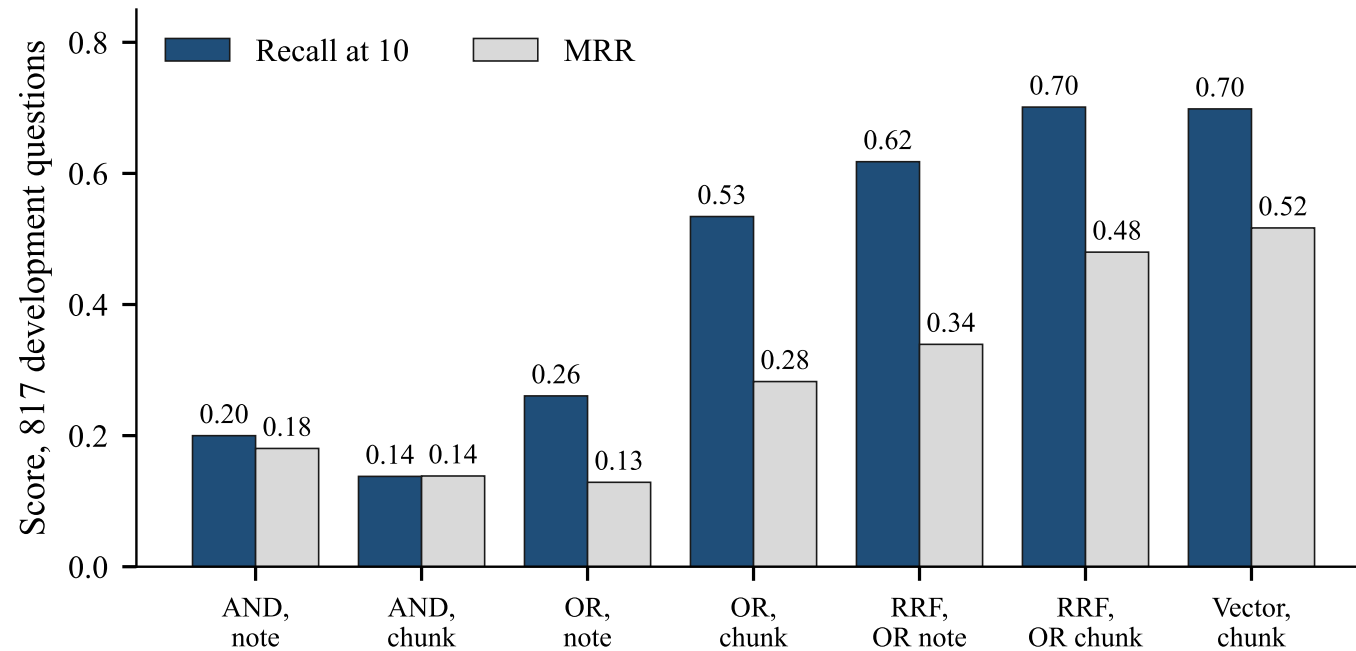
Example: one answering note, found at rank 3, gives recall 1, MRR 0.33 and nDCG 0.50.

¹Voorhees, The TREC-8 question answering track report, TREC 1999.

²Järvelin and Kekäläinen, Cumulated gain-based evaluation of IR techniques, ACM TOIS 20(4), 2002.

Fixing keywords does **not** change the winner

Result 1



AND to OR. Switching keyword search from AND to OR lifts it from 0.138 to 0.534.

Rank fusion. Mixing keywords and vectors ties vectors on recall (0.701 vs 0.698) and lowers the top-rank quality.¹

Net effect. On this corpus, keywords add nothing that vectors miss often enough to help.

¹Reciprocal rank fusion: Cormack, Clarke and Büttcher, SIGIR 2009.

Return **chunks**, not whole notes

Result 2

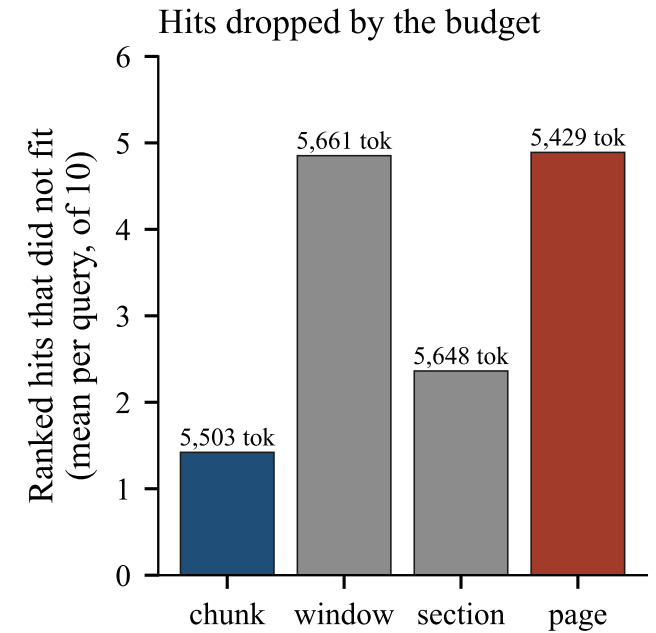
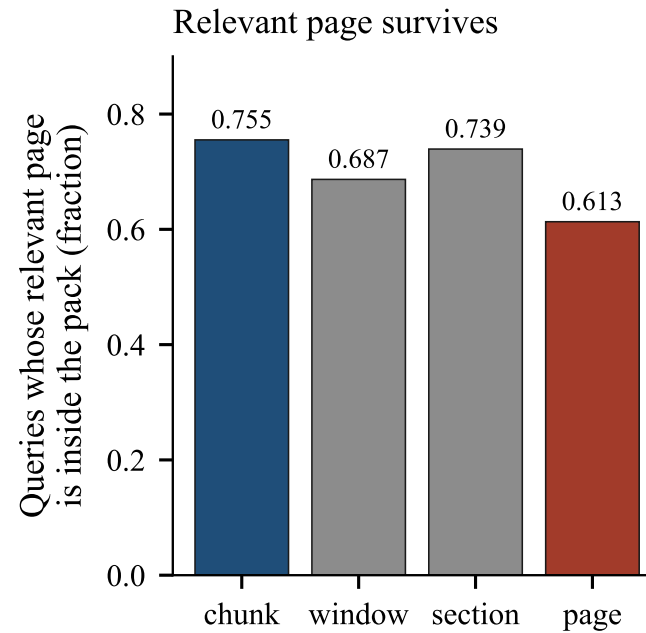
Under a 6,000-token budget, how often is the right note inside what the model gets?

0.755
chunks

0.613
whole notes

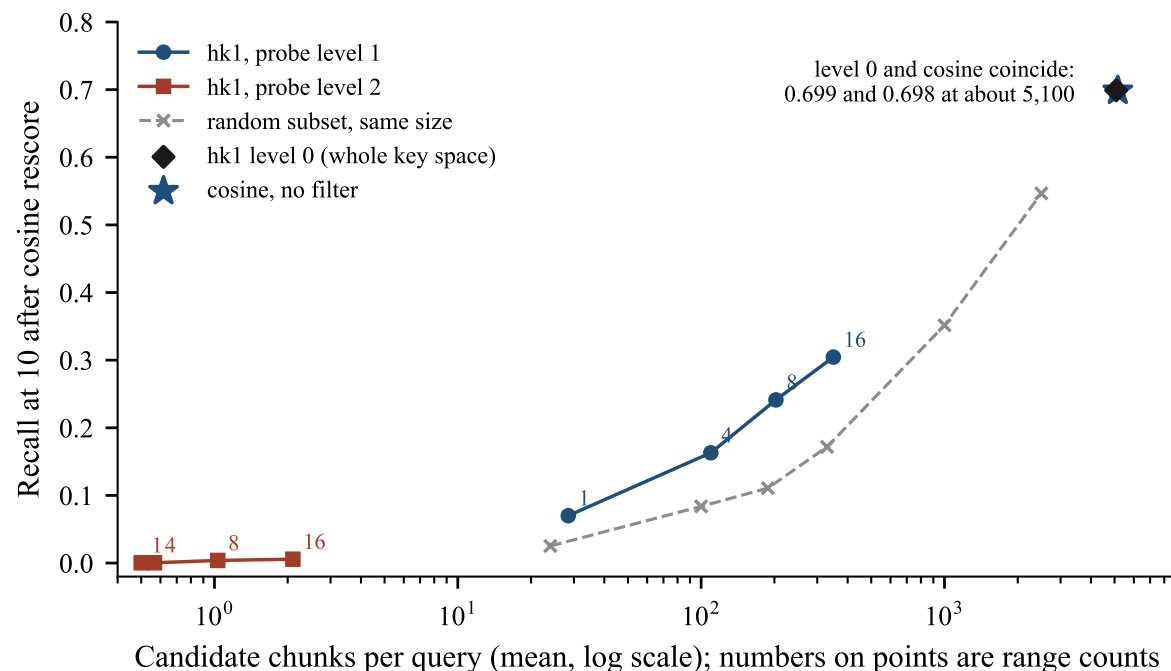
Same tokens spent; smaller units let more distinct notes fit. No question was better served by whole notes.

6,000 tokens is gbrain's default. No unit beats chunks at any budget tried, from 1,000 (0.410 vs 0.157) to 24,000 (0.767 vs 0.737).



A Hilbert key is a **label**, not an index

Result 3



0.304

best Hilbert-key filter: level 1, 16 ranges

0.698

plain vector search

The key does carry signal: about twice a random filter of the same size. It does not come close to exhaustive search.

Per token the model reads, hk1 is half as efficient: a relevant note per 1k delivered tokens 0.068 vs 0.137. Per token of candidates read it is 8× better than reading everything.

How to read the chart

hk1 key: each chunk's vector becomes one 64-bit text key: 8 random projections, each cut into 256 steps, ordered along a Hilbert curve so that nearby keys should mean nearby vectors.

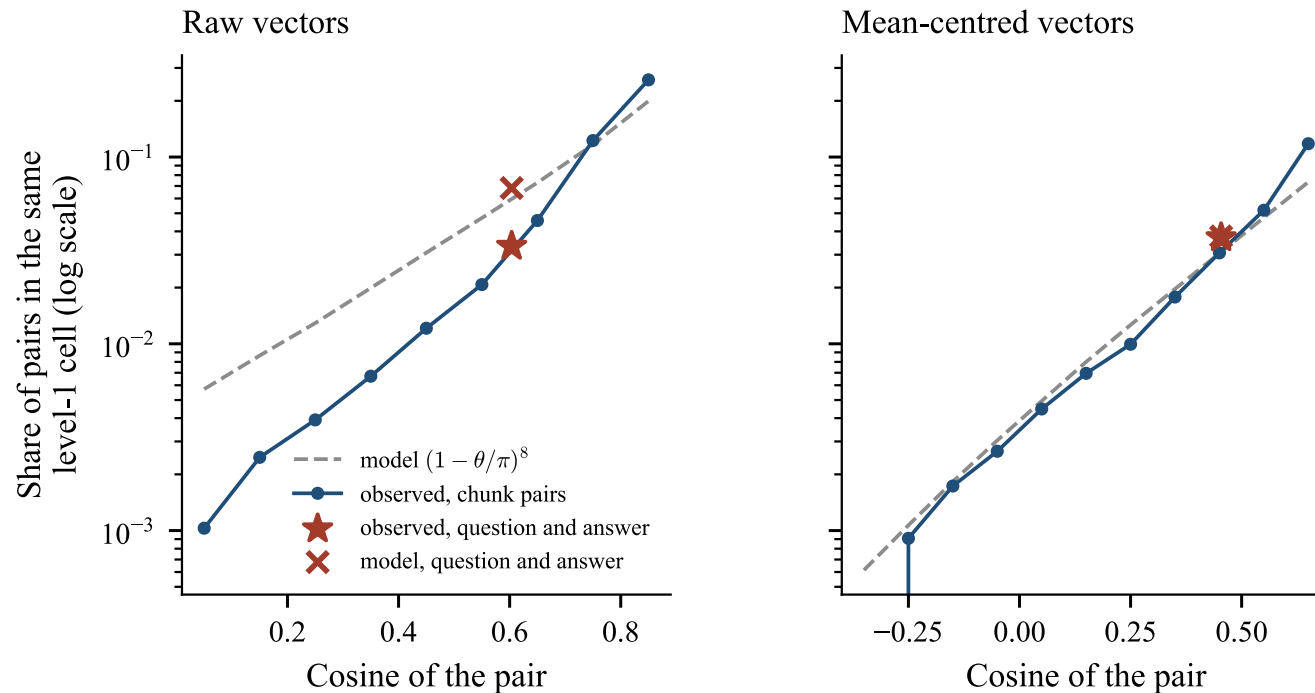
Probe level: how many leading bytes of a chunk's key must equal the question's. Level 1 (8 bits) splits the 5,079 keyed chunks into 256 cells of about 20 each; level 2 (16 bits) into 65,536 cells, almost all empty; level 0 keeps every chunk.

Range count (number on a point): the question's own cell plus up to $r - 1$ neighbouring cells: 1, 4, 8 or 16. Level 1 grows from 28 to 349 candidates per question; level 2 stays below 3.

Score of a point: the candidates are re-ranked by exact cosine and scored by recall at 10. Grey crosses use the same number of random chunks (the floor); the star is cosine over all chunks (the ceiling).

Why the key fails: it is an 8-bit hash

Result 3



SimHash. The first level of the key is the sign pattern of 8 random projections: an 8-bit SimHash.¹

Narrow cone. Embeddings crowd into a narrow cone; after removing the mean direction, hashing theory predicts the measured collisions: 0.037 observed vs 0.038 predicted.²

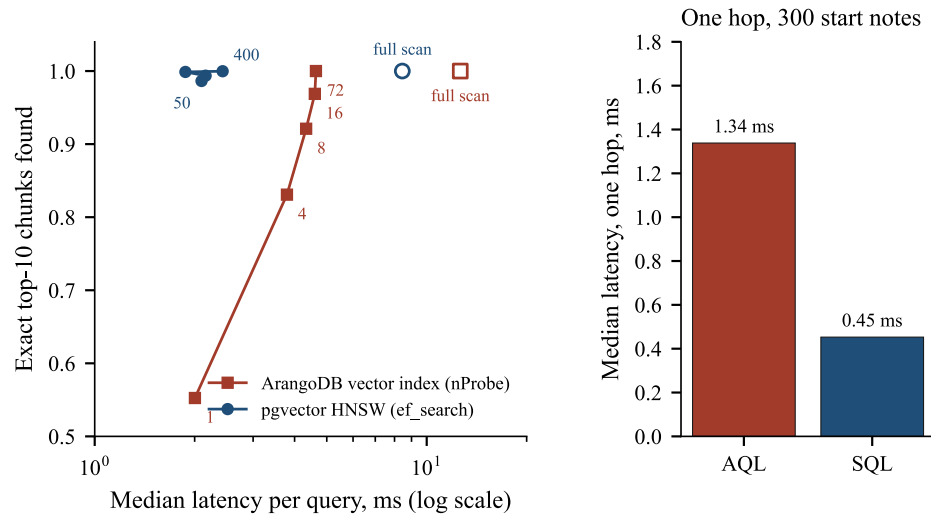
Too far apart. A question and its answer are too far apart for one key: about 71 independent keys would be needed for 90% recall.

¹Charikar, STOC 2002; Indyk and Motwani, STOC 1998.

²Ethayarajh, EMNLP 2019; Mu and Viswanath, ICLR 2018.

Postgres is enough

Result 4



Vector search. pgvector 2.1 ms vs ArangoDB 4.6 ms at matched quality.

Link expansion. Identical neighbours, 0.45 ms in SQL vs 1.34 ms in AQL.

Scale. At 200,000 vectors pgvector still answers in 0.80 ms.

Measured on PostgreSQL 16.15 and ArangoDB 3.12.10. Re-run on 18.6 and 3.12.12: same quality, same order (1.2 vs 4.2 ms; hop 0.16 vs 1.13 ms).

Why ArangoDB first, and what the other engines did

ArangoDB: a native graph of documents plus a vector index, so both operations the knowledge base needs, vector search and link expansion, are measured in one engine. It already runs on this machine for a genealogy database.

SQLite + sqlite-vec: a single file, exact scan only, same SQL for links. Measured in process: 5.9 ms, same recall. gbrain cannot run on it.¹

Turso (libSQL): a SQLite fork with a vector index. Measured in process: 8.8 ms, same recall. Not a gbrain backend.²

MongoDB: Community 9.0.2 with mongot, measured: 9.1 ms vs pgvector 1.5 ms, same recall; links 0.63 vs 0.14 ms.³

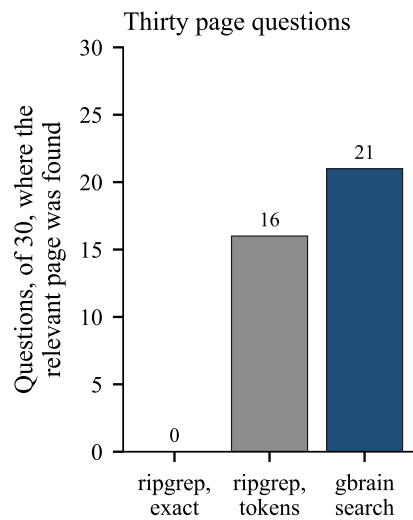
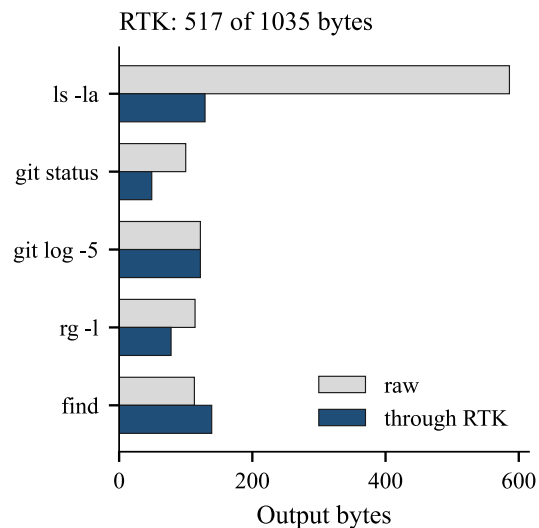
¹sqlite-vec, github.com/asg017/sqlite-vec; gbrain supports only Postgres and PGLite.

²libSQL, github.com/tursodatabase/libsql.

³MongoDB, Vector Search on self-managed deployments, mongodb.com/docs/search/self-managed.

Agent side: first observations

First look



RTK. Cut five shell commands from 1,035 to 517 bytes, but made one of them longer.

gbrain search. On 30 questions it found the right note 21 times; exact text search found it 0 times.

Local 27B model: a 32K window used 2.1× the tokens of 8K with no measured gain; 128K did not fit in 32 GB. Cloud agents were not run.

What RTK is

What it does: a small command-line proxy between the AI agent and the shell. A hook in Cursor or Claude Code rewrites a command such as `git status` into `rtk git status`, runs it, and hands the agent a condensed version of the output, so the model reads fewer tokens.¹

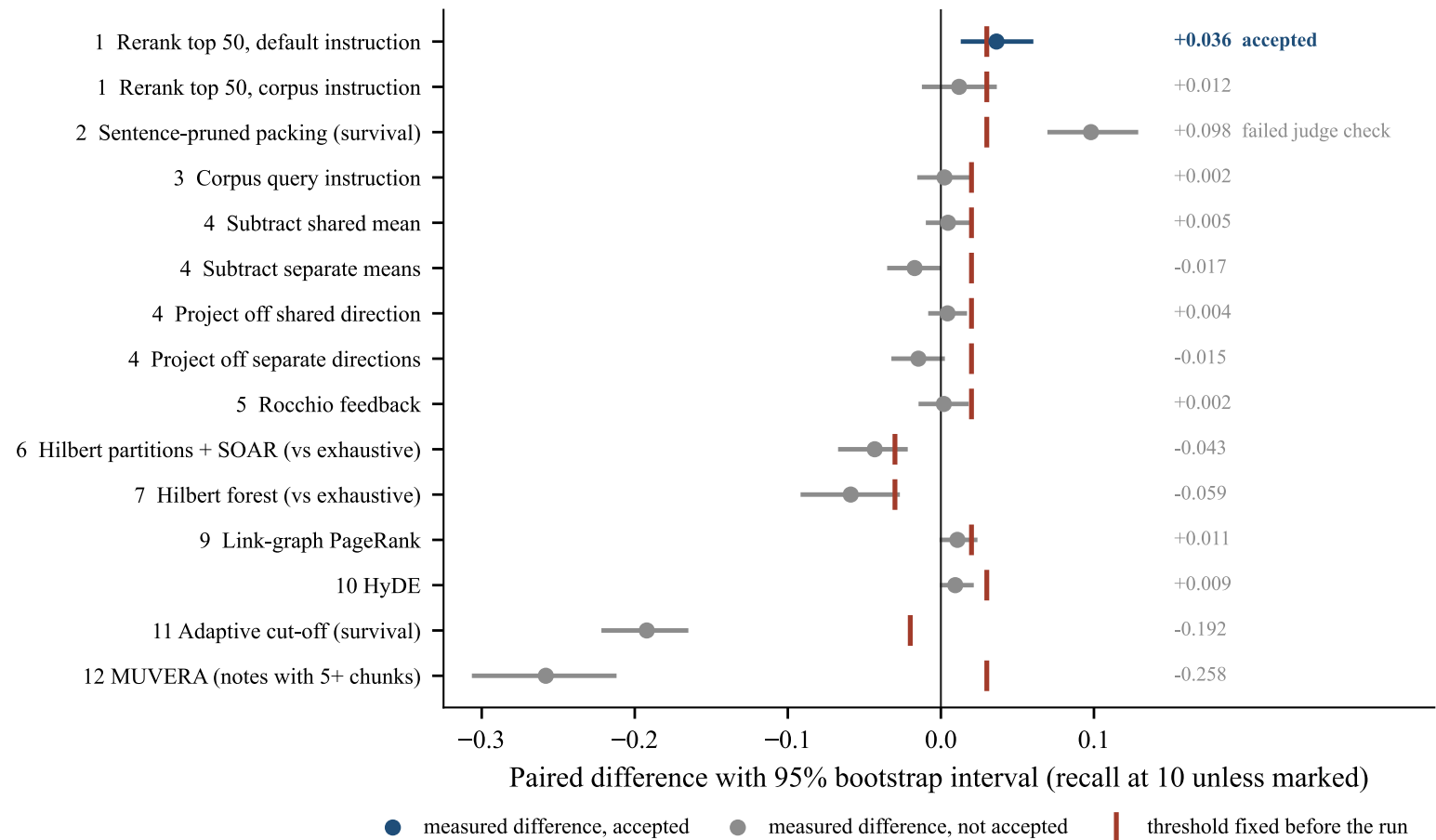
What it does not touch: only shell commands pass through it. The agent's own file-read and search tools, MCP calls such as `gbrain`, and the model's replies are not filtered. If the hook fails, the raw command runs.

How it was measured: five fixed commands, each run raw and through RTK, counting output bytes: `ls -la` 586 → 129, `git status` 100 → 49, `rg` 114 → 78, `git log` unchanged, `find` 113 → 139 (longer).

¹RTK 0.51.0, www.rtk-ai.app; Cursor hook registered with `rtk init -g --agent cursor`.

Twelve ideas from recent papers, tested

Exploratory



+0.036

recall from reranking the vector top 50 — the only idea accepted

56 near-misses move into the top ten; cost 4.4 s per question.

Sentence-level packing kept more notes but a blind judge found the text answered less often (77% vs 82%), so it failed.

More relevant notes per token? **Not with these**

Exploratory

	Method	Change in survival
M1	Relevance per token, greedy fill	-0.054
M2	Budgeted coverage (submodular)	-0.333
M3	Diversity: maximal marginal relevance	-0.009
M4	Diversity: determinantal point process	-0.006
M5	Heat diffusion over links	-0.021
M6	Manifold ranking	-0.017
M7	Advection–diffusion over links	-0.069

Survival: a relevant note is in the pack, averaged over budgets of 1,000 to 6,000 tokens; confirmation half, 408 questions. None reached the pre-registered +0.02.

-8%

tokens from a calibrated early stop (M9), survival unchanged

Conformal risk control picks the stop: 5,040 instead of 5,486 tokens per question, survival 0.760 vs 0.767. The bar was 20%, so it is not accepted.

Why. Every method's best setting was the one closest to plain cosine order. Repetition and links are not what loses notes here.

Scoreboard so far

Question	Winner	Measured
Keywords or vectors?	vectors	recall 0.698 vs 0.200
What to hand the model?	chunks	right note kept 0.755 vs 0.613 at equal tokens
Hybrid search?	no gain	0.701 vs 0.698
Hilbert key instead of vector search?	no	0.304 vs 0.698
Hilbert key, per token the model reads?	cosine	0.068 vs 0.137 relevant per 1k delivered tokens
Hilbert key, per token of candidates read?	hk1	8× reading everything; about 2× random
Second database engine?	Postgres	faster at both operations tested
Smarter packing per token?	plain cosine	8 methods tested, none better; an early stop saves 8% of tokens
Reranker?	yes, on vectors	+0.036 recall for 4.4 s
Holds on unseen questions?	yes	held-out recall 0.725
Sentence packs on unseen questions?	yes	right note kept 0.897 vs 0.752, fewer tokens
Sentence packs answer the question?	chunks	judged sufficient 0.713 vs 0.793, 7 vs 19 questions
Hilbert key as a facet map?	no	found 0.182 vs 0.479, with 7.5× the tokens per answer
Reranker null on unseen questions?	underpowered	power 66%; with judge labels +0.119

These are *measured findings, not decisions*.

PART 3

Every combination



Every combination, then **unseen questions**

Exploratory

0.897

held-out questions with a relevant note in the pack, packing the best sentences of the top 30 notes (chunks: 0.752)

124 questions favour sentence packs, 5 favour chunks; 1,517 fewer tokens. A pre-registered judge found them sufficient to answer less often (0.713 vs 0.793 on 150 questions), so this is reaching the note, not answering.

1,020 combinations. Eight switches crossed on 817 questions; each switch scored by its average contribution (Shapley value).

Reranker. Largest recall gain on development questions (+0.050) for 4.4 s; on unseen questions the best combination's gain shrank to +0.016, not significant.

Hilbert key filter. -0.367 recall in every combination.

The same switches, **measured again**

Exploratory

Best sentence. Recall at 10 0.660 against 0.698.

Seed clamp. Recall changed by -0.001 . The original top note stayed in the top ten.

Forward push. 0.720 against 0.713, and the walk is slower.

Reranker on exact ties. 0.16 s against 4.56 s.
Confirmation MRR stays inside the always-on interval.

77%

fewer tokens when a separate checker stops the pack

The 27B judge calls those packs sufficient for 0.687 of 150 questions, against 0.793 for the full chunk pack.

Winning sentence and its headings. 466 tokens, sufficient for 0.347.

Cover both pages. Set recall 0.487 on the 60 two-page questions, above 0.479.

The held-out questions stay sealed.

Thank **you**

Paper, measured results and every script:

github.com/guanchzhou/hilbert-paper

The notes themselves are private and are not in the repository.

